SDL Practices

exercises

Henrik Kramselund
hlk@zencurity.com

October 14, 2026



Note: exercises marked with ⚠ are considered important. These contain subjects that are essential for the course. Even if you don't work through the exercise, you might want to know the subjects covered by these.

Exercises marked with ℹ are considered optional. These contain subjects that are related to the course, but less important. You may want to browse these and if interested work through them. They may require more time than we have available during the course.

Contents

1 ⚠ Prepare your workstation 15 min 2

2 ⚠ Git tutorials - 15min 3

3 ⚠ Install and Run Kubernetes 45 min 5

4 ℹ Run OWASP Juice Shop 45 min 8

5 ℹ Setup JuiceShop and proxy - up to 20min 11

6 ℹ CIS benchmark scan with kube-bench up to 30 min 13

7 ℹ Trivy scan of container images in k3s up to 40 min 14

8 ⚠ JuiceShop Attacks 60min 16

9 ⚠ Truncate and Encoding Attacks JuiceShop up to 40min 18

10 ⚠ JuiceShop Login 15 min 20

11 ⚠ Git hook 30 min 21

12 ⚠ Small programs with data types 15min 23

13 ⚠ Pointers and Structure padding 30min 24

14 ⚠ Trying PMD static analysis 30 min 25

15 ⚠ RBAC Access permissions on GitHub 15 min 26

16 ⚠ Research CSRF protection in Django and OWASP 15 min 27

17 ⚠ Kyverno Playground 15 min 28

18 ⚠ Threat-model your pipeline 15 min 30

19 ⚠ npm audit and osv-scanner on Juice Shop 30 min 31

20 ℹ Identify Session Tokens 30 min 33

© license CC BY 3.0. 2026 Henrik Kramselund

21 ℹ Postman API Client 20 min 34

22 ℹ PHP Passwords 30 min 35

23 ℹ Django String Handling 20min 37

24 ℹ Django ORM 20 min 38

25 ℹ Django email validation 30 min 39

26 ℹ Try American fuzzy lop up to 60min 40

27 ℹ Real Vulnerabilities up to 30min 41

28 ℹ Security.txt 15 min 42

29 ℹ Securing the JuiceShop 43

30 ℹ Lynis Auditing, System hardening, and Compliance testing 20min 44

© license CC BY 3.0. 2026 Henrik Kramselund

Preface

This material is prepared for use in SDL Practices course and was prepared by Henrik Kramselund, https://www.zencurity.com. It describes the networking setup and applications for trainings and courses where hands-on exercises are needed.

Further presentations are used which are available as PDF from `https://codeberg.org/kramse` Look for SDL-practices-exercises in the repo security-courses.

These exercises are expected to be performed in a training setting with network connected systems. The exercises use a number of tools which can be copied and reused after training. A lot is described about setting up your workstation in the repo

https://codeberg.org/kramse/kramse-labs

Prerequisites

This material expects that participants have a working knowledge of TCP/IP from a user perspective. Basic concepts such as web site addresses and email should be known as well as IP-addresses, ports and common protocols like HTTP/HTTPS.

Have fun and learn

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise content

Most exercises follow the same procedure and have the following content:

- **Objective:** What is the exercise about, the objective
- **Purpose:** What is to be the expected outcome and goal of doing this exercise
- **Suggested method:** suggest a way to get started
- **Hints:** one or more hints and tips or even description how to do the actual exercises
- **Solution:** one possible solution is specified
- **Discussion:** Further things to note about the exercises, things to remember and discuss

Please note that the method and contents are similar to real-life scenarios and do not detail every step of doing the exercises. Entering commands directly from a book only teaches typing, while the exercises are designed to help you become able to learn and actually research solutions.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 1

⚠ Prepare your workstation 15 min

Objective:
Get your workstation ready for the hacker lab.

Purpose:
We will be running multiple tools

Suggested method:

- Create a folder for files from this course

Hints:
There are Git repositories on https://codeberg.org/kramse/

Solution:
When you have created a folder for files you are done, the rest can wait a bit.

Discussion:
Do we need to run local VMs, can we run Docker or Kubernetes instead – yes for some exercises, no for others.

You can work together in groups and only one workstation might have applications running.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 2

⚠ Git tutorials - 15min



Objective:
Try the program Git locally on your workstation

Purpose:
Running Git will allow you to clone repositories from others easily. This is a great way to get new software packages, and share your own.

Git is the name of the tool, and Codeberg and GitHub are popular sites for hosting git repositories.

Suggested method:
Run the program from your Linux VM. You can also clone from your Windows or Mac OS X computer. Multiple graphical front-end programs exist too.

First make sure your system is updated, as root run:

```
sudo apt update && sudo apt -y upgrade
```

You should reboot if the kernel is upgraded :-)

Second make sure your system has Git, ansible and my playbooks: (as root run, or with sudo as shown)

```
sudo apt -y install ansible git
```

Most important are Git clone and pull:

```
user@Projects:tt$ git clone https://codeberg.org/kramse/kramse-labs.git
Cloning into 'kramse-labs'...
remote: Enumerating objects: 283, done.
remote: Total 283 (delta 0), reused 0 (delta 0), pack-reused 283
Receiving objects: 100% (283/283), 215.04 KiB | 898.00 KiB/s, done.
Resolving deltas: 100% (145/145), done.

user@Projects:tt$ cd kramse-labs/

user@Projects:kramse-labs$ ls
LICENSE  README.md  core-net-lab  lab-network  suricatazeek  work-station
user@Projects:kramse-labs$ git pull
Already up to date.
user@Projects:kramse-labs$ ls
core-net-lab/  kubernetes/   suricatazeek/  work-station/  README.md
docker-install/  lab-network/  web-security/  LICENSE
```

If you want to run Ansible playbooks, then run it with:

```
ansible-playbook -v example.yml
```

Hints:
Browse the Git tutorials on https://git-scm.com/docs/gittutorial
and https://guides.codeberg.org/activities/hello-world/

We will not do the whole tutorials within 15 minutes, but get an idea of the command line, and see examples. Refer back to these tutorials when needed or do them at home.

Note: you don't need an account on Codeberg or Github to download/clone repositories, but having an account allows you to save repositories yourself and is recommended.

Solution:
When you have tried the tool and seen the tutorials you are done.

Discussion:
Before Git there have been a range of version control systems,
see https://en.wikipedia.org/wiki/Version_control for more details.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 3

⚠ Install and Run Kubernetes 45 min

This is a fairly large exercise, and part of this course

Objective:

We will use containers, images, Linux for many exercises.

Kubernetes is an easy and standardized way to run many applications, which will allow us to deploy applications for testing quickly with just a few commands.

Purpose:

The course standardizes on Kubernetes k3s, so that everyone ends up on the same Kubernetes distribution and the exercise manifests should work the same for everyone.

Run different applications across your computers in a common way, to see the same versions and setups with minimal differences.

Suggested method:

This exercise has two install paths and one shared check at the end.

We expect a Kubernetes install for each team, up to three members per team.

Read this first:

If you already have Docker, Podman, VirtualBox, VMware, Hyper-V, WSL2 distros, KVM/libvirt, or any other container or virtualization tooling installed and want to keep it skip straight to Advanced Install below. Don't install Rancher Desktop on top of an existing setup; go to Advanced Install instead.

If your laptop is clean no containers, no VMs, nothing like that use Quick Start which uses Rancher Desktop.

Requirements are not especially high for Rancher Desktop, mostly you need:
64-bit CPU with hardware virtualization (Intel VT-x or AMD-V), enabled in BIOS/UEFI.

Mac OS X laptops require macOS 13 (Ventura) or newer. Apple Silicon (aarch64) or Intel (`x86_64`) CPU with VT-x. Recommended: 8 GB RAM, 4 CPU cores.

Quick Start (nothing installed yet)

This is the expected path for most of you.

1. **Uninstall leftovers first, if any.** If you've ever installed Docker Desktop, VirtualBox, or WSL distros for another course and aren't using them anymore, remove them before continuing running two virtualization stacks side by side is a common source of very confusing errors.
2. **Install Rancher Desktop** `https://rancherdesktop.io` . One installer per OS (Windows / macOS / Linux), free and open source, no license or account needed.
3. On first launch - use the following settings: `Enable Kubernetes: on` (is probably on by default) and `Engine: containerd` (select this, probably docker is default)
4. Wait for Rancher Desktop to show the cluster as running (first launch takes a few minutes it's downloading k3s).
5. Continue to Check Your Environment below.

© license CC BY 3.0. 2026 Henrik Kramselund

—

Advanced Install (you already have something)
This path is for people who already have working tooling and don't want Rancher Desktop's VM layer on top of it. It's higher-level than Quick Start you're expected to be comfortable following official docs for your specific tool.

Already have Docker / Docker Engine installed and running?
Install k3d `https://k3d.io` (k3s running inside Docker containers you already have). No new VM, reuses your existing Docker install.

```
k3d cluster create softsec
```

On Linux, with KVM/libvirt available (or no virtualization tooling at all)?
Install k3s `https://k3s.io` directly on the host. On Linux, Kubernetes doesn't need a VM at all the host is the container host.

```
curl -sfL https://get.k3s.io | sh -
```

On Windows or Mac, with WSL2 or another VM already set up that you want to keep using?
Install k3s inside that existing VM/WSL2 distro directly, following the k3s docs at `https://docs.k3s.io`, rather than adding Rancher Desktop's own VM alongside it.

Already running Docker Desktop's own Kubernetes, minikube, or kind? These work differently enough from k3s (different defaults, sometimes different Kubernetes version) that we'd still recommend adding k3s/k3d alongside for this course, so your environment matches the exercises. Keep your existing tool for anything else you use it for.

If your situation doesn't match any of the above, let's discuss.

Check Your Environment
Run once your cluster (Rancher Desktop or Advanced) is up.

1. Confirm `kubectl` can see your cluster:

```
kubectl get nodes
```

Expect one node, status `Ready`. Example output:

```
hlk@alice:~$ kubectl get nodes
NAME                   STATUS   ROLES           AGE   VERSION
lima-rancher-desktop   Ready    control-plane   17m   v1.36.3+k3s1
```

2. Run a test pod:

```
kubectl run hello --image=nginx --restart=Never
kubectl get pods
```

Expect `hello` to reach `Running` within about a minute (`kubectl get pods -w` to watch it live). Example output:

© license CC BY 3.0. 2026 Henrik Kramselund

```
hlk@alice:~$ kubectl run hello --image=nginx --restart=Never
// wait a bit and then
hlk@alice:~$ kubectl get pods
NAME    READY   STATUS    RESTARTS   AGE
hello   1/1     Running   0          5m45s
```

3. Confirm you can reach it:

```
kubectl port-forward pod/hello 8080:80
```

then open `http://localhost:8080` in a browser you should see the nginx welcome page. Ctrl+C to stop the port-forward. Example output:

```
hlk@alice:~$ kubectl port-forward pod/hello 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
Handling connection for 8080
Handling connection for 8080
```

4. Clean up:

```
kubectl delete pod hello
```

If all four steps worked, your environment is ready for the exercises. If step 1 fails, your cluster isn't running go back to whichever install path you used. If steps 24 fail but step 1 worked, flag it before the first exercise session.

Hints:
If you are low on memory and disk you might have problems.

Solution:
When you have a k8s/k3s cluster running as a team, then we are good.

Discussion:
Linux is free and everywhere. The tools we will run in this course are made for Unix, so they run great on Linux.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 4

ℹ Run OWASP Juice Shop 45 min



Objective:
Let's try starting the OWASP Juice Shop

Purpose:
We will be doing some web hacking where you will be the hacker. There will be an application we try to hack, designed to optimise your learning.

It is named JuiceShop which is written in JavaScript

Suggested method:
Go to https://github.com/juice-shop/juice-shop

You need to have browsers and a proxy, plus a basic knowledge of HTTP.

Read the instructions for running juice-shop - **docker or Kubernetes is a simple way** and I often recommend that.

Running with Kubernetes follows.

First check your cluster: is it ready, and can you connect?

```
hlk@alice:~$ kubectl get node
NAME                   STATUS   ROLES           AGE   VERSION
lima-rancher-desktop   Ready    control-plane   13d   v1.36.3+k3s1
hlk@alice:~$
```

© license CC BY 3.0. 2026 Henrik Kramselund

Create a Deployment, name the file `juice-shop.yml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: juice-shop
  labels:
    app: juice-shop
spec:
  replicas: 1
  selector:
    matchLabels:
      app: juice-shop
  template:
    metadata:
      labels:
        app: juice-shop
    spec:
      containers:
        - name: juice-shop
          image: bkimminich/juice-shop:latest
          ports:
            - containerPort: 3000
---
apiVersion: v1
kind: Service
metadata:
  name: juice-shop
spec:
  selector:
    app: juice-shop
  ports:
    - port: 3000
      targetPort: 3000
  type: ClusterIP
```

This file can also be downloaded from Git kramse-labs in the directory `kubernetes`
`git clone https://codeberg.org/kramse/kramse-labs`

Run the deployment:

```
hlk@alice:kubernetes$ kubectl apply -f juice-shop.yml
deployment.apps/juice-shop created
service/juice-shop created
hlk@alice:kubernetes$
```

© license CC BY 3.0. 2026 Henrik Kramselund

After a short while you should have a running pod/container and deployment:

```
hlk@alice:kubernetes$ kubectl get pods
NAME                           READY   STATUS    RESTARTS   AGE
hello                          0/1     Unknown   0          13d
juice-shop-86c67bcdf4-lf87x    1/1     Running   0          2m27s
hlk@alice:kubernetes$ kubectl get deployments
NAME         READY   UP-TO-DATE   AVAILABLE   AGE
juice-shop   1/1     1            1           2m38s
```

Wait for Running, then reach it using port forward:

```
hlk@laptop:~$ kubectl port-forward svc/juice-shop 3000:3000
```

Note: This needs to be done regularly. Familiarise yourself with `kubectl apply`, `kubectl delete` and `kubectl port-forward`

Check using: http://127.0.0.1:3000

Hints:
The application is very modern, very similar to real applications.

JuiceShop can be run as a Docker container, and sometimes running it on Kali is the easiest learning environment.

Solution:
When you have a running Juice Shop web application in your team, then we are good.

Discussion:
It has lots of security problems which can be used for learning hacking, and thereby how to secure your applications. It is related to the OWASP.org Open Worldwide Application Security Project which also has a lot of resources.

Sources:
`https://github.com/juice-shop/juice-shop`
`https://owasp.org/www-project-juice-shop/`

It is recommended to buy the Pwning OWASP Juice Shop Official companion guide to the OWASP Juice Shop from https://leanpub.com/juice-shop - suggested price USD 5.99

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 5

❶ Setup JuiceShop and proxy - up to 20min

Objective:
Run JuiceShop with Burp proxy.

Start JuiceShop and make sure it works, visit using browser.

Then add a web proxy in-between. We will use Burp suite which is a commercial product, in the community edition.

Purpose:
We will learn more about web applications as they are a huge part of the applications used in enterprises and on the internet. Most mobile apps are also web applications in disguise.

By inserting a web proxy we can inspect the data being sent between browsers and the application.

The Burp proxy is an advanced tool! Don't be scared, we will use small parts at different times.

Suggested method:
You need to have browsers and a proxy, plus a basic knowledge of HTTP.

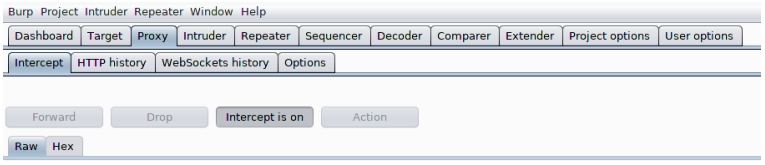
If you could install Firefox it would be great, and we will use the free version of Burp Suite, so please make sure you can run Java and download the free version from Portswigger from:

https://portswigger.net/burp/communitydownload

follow the Getting Started instructions at:
https://support.portswigger.net/customer/portal/articles/1816883-getting-started-with-burp-suite

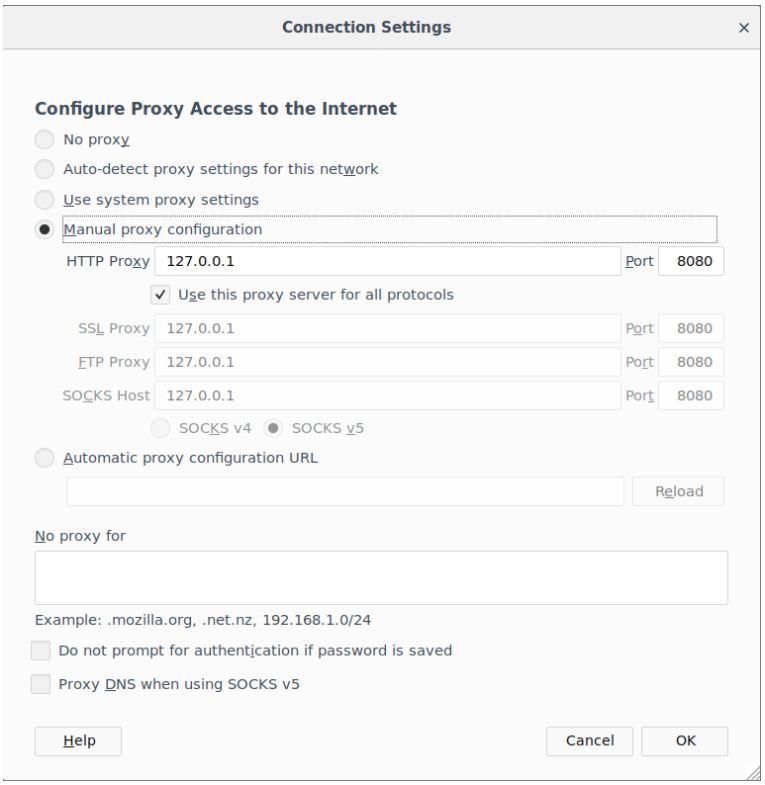
Hints:
Recommend running Burp on the default address and port 127.0.0.1 port 8080.

Note: Burp by default has `intercept is on` in the Proxy tab, press the button to allow data to flow.



© license CC BY 3.0. 2026 Henrik Kramselund

Then setting it as proxy in Firefox:



After setting up proxy, you can visit `http://burp` and get a CA certificate that can be installed, making it easier to run against HTTPS sites.

The newest versions of Burp include a browser, making it much easier to run the tasks, pre-configured with proxy.

Firefox prevents hijacking localhost by default! Use about:config and set flag

`network.proxy.allow_hijacking_localhost` set to `TRUE`

Solution:

When web sites and servers start popping up in the Target tab, showing the requests and responses - you are done.

Your browser will alert you when visiting TLS enabled sites, HTTPS certificates do not match, as Burp is doing a person-in-the-middle. You need to select advanced and allow this to continue.

Discussion:

Since Burp is often updated I use a small script for starting Burp which I save in `~/bin/burp` – don't forget to add to PATH and `chmod +x bin/burp`.

```
#! /bin/sh
DIRNAME=`dirname $0`
BURP=`ls -1tra $DIRNAME/burp*.jar | tail -1`
java -jar -Xmx6g $BURP &
```

When running in production testing real sites, I typically increase the memory available using JDK / Java settings like `-Xmx16g`

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 6

ℹ CIS benchmark scan with kube-bench up to 30 min

Objective:
We will run kube-bench against our local k3s cluster to check it against the CIS Kubernetes Benchmark – a checklist of security configuration recommendations for Kubernetes clusters.

Purpose:
See what a configuration/hardening scanner looks like, as opposed to a vulnerability scanner like Trivy from another exercise. kube-bench doesn't look for known CVEs – it checks whether the cluster itself (API server, kubelet, etcd, etc.) is configured according to security best practice.

Suggested method:
Run kube-bench as a Job inside your k3s cluster using `kubectl`, since that gives it access to the node's config files and binaries needed to check things like file permissions. Then read through the output.

Hints:
The kube-bench project publishes ready-made Kubernetes manifests to run it as a Job:

```
hlk@laptop:~$ kubectl apply -f https://raw.githubusercontent.com/aquasecurity/kube-bench/main/job.yaml
```

Wait for it to finish, then read the logs:

```
hlk@laptop:~$ kubectl get pods | grep kube-bench
hlk@laptop:~$ kubectl logs job/kube-bench
```

k3s bundles components a bit differently than a "vanilla" kubeadm cluster (e.g. no separate etcd pod, different config file paths), so don't be surprised if some checks come back as `[INFO]` or `[WARN]` "not applicable" rather than a clean pass/fail – there is a k3s-specific config/target you can look for in the kube-bench docs if you want more accurate results. Clean up afterwards:

```
hlk@laptop:~$ kubectl delete -f https://raw.githubusercontent.com/aquasecurity/kube-bench/main/job.yaml
```

Solution:
You are done once you can point to at least one `[FAIL]` or `[WARN]` result in the output, explain in your own words what the check was actually verifying, and say whether it's something you'd realistically fix on a student laptop cluster or only on a production cluster.

Discussion:
Look up the Trivy scanner, and compare kube-bench to Trivy: Trivy tells you what known-vulnerable software is running; kube-bench tells you whether the cluster is configured safely, regardless of software versions. Both are needed. Discuss:

- Why a perfectly patched cluster can still be insecure if it's misconfigured (e.g. anonymous API access enabled)
- Why some checks are unrealistic to fully satisfy on a local dev cluster (e.g. those about running on dedicated, hardened hosts) but matter a lot in production
- How this fits into a broader "Secure by Design" pipeline: SCA (Trivy) + config auditing (kube-bench) + your own code's static analysis (PMD)

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 7

❶ Trivy scan of container images in k3s up to 40 min

Objective:
We will use Trivy to scan the container images actually running in our local k3s cluster (the one set up with Rancher Desktop earlier in the course) for known vulnerabilities. We use `kubectl` to find out which images are running, and Trivy to scan them.

Purpose:
Get hands-on experience with a real vulnerability scanner and see how it maps found issues back to:

- CVE identifiers
- CWE categories (compare with the OWASP Top 10 and CWE Top 25 discussed in class)
- Severity ratings (CRITICAL / HIGH / MEDIUM / LOW)

This is your first taste of Software Composition Analysis (SCA) – scanning not code you wrote, but the third-party layers you depend on (base images, OS packages, libraries).

Suggested method:
Make sure your k3s cluster (Rancher Desktop) is running and that you have something deployed in it – e.g. NGINX from an earlier exercise. If nothing is deployed yet, deploy something first. Install Trivy on your workstation (not inside the cluster). Follow the instructions from
`https://trivy.dev/docs/latest/getting-started/installation/`

Use `kubectl` to list which images are actually running, then scan those images directly with Trivy.

Hints:
Find the images currently running in your cluster:

```
hlk@laptop:~$ kubectl get pods -A -o jsonpath="{.items[*].spec.containers[*].image}"    | tr -
s ' ' '
n' | sort -u
```

Scan a single image:

```
hlk@laptop:~$ trivy image bkimminich/juice-shop:latest
```

Trivy has a dedicated Kubernetes mode that talks to your cluster directly via your kubeconfig, and will enumerate and scan all running workloads for you – try it once you have scanned a single image manually:

```
hlk@laptop:~$ trivy k8s --report summary cluster
```

If you want vulnerability data only (skip secrets/misconfig scanning, faster):

```
hlk@laptop:~$ trivy image --scanners vuln bkimminich/juice-shop:latest
```

© license CC BY 3.0. 2026 Henrik Kramselund

Try filtering to only show what matters for patching:

```
hlk@laptop:~$ trivy image --severity CRITICAL,HIGH bkimminich/juice-shop:latest
```

Solution:
A finished run produces a table (or JSON, with `-f json`) listing, for each vulnerable package, the installed version, the fixed version (if one exists), the CVE ID, and severity. You are done once you can:

- Explain what one specific CVE in the output actually is (look it up)
- State whether a fix is available for it
- Say whether it applies to the OS layer (e.g. Debian/Alpine packages) or the application layer (e.g. npm packages)

Discussion:
Trivy will usually report far more vulnerabilities than you can realistically fix – this is normal, and mirrors real-world alert fatigue in vulnerability management. Discuss:

- Why base image choice (e.g. `alpine` vs `debian` vs `distroless`) drastically changes the vulnerability count
- Why "no fix available" is common and what you'd do about it in practice (accept risk, find an alternative package, mitigate elsewhere)
- How this connects to CI/CD – scanning should happen before deploy, not just ad-hoc like we just did
- The difference between scanning an image (Trivy) and scanning your own source code for weaknesses (static analysis, e.g. PMD from an earlier exercise) – Trivy tells you about vulnerabilities in dependencies, not bugs in your own code

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 8

⚠ JuiceShop Attacks 60min



Objective:
Hack a web application!

Try a few attacks in the JuiceShop with web proxy

> The OWASP Juice Shop is a pure web application implemented in JavaScript. In the frontend the popular AngularJS framework is used to create a so-called Single Page Application. The user interface layout is provided by Twitter's Bootstrap framework - which works nicely in combination with AngularJS. JavaScript is also used in the backend as the exclusive programming language: An Express application hosted in a Node.js server delivers the client-side code to the browser. It also provides the necessary backend functionality to the client via a RESTful API.
>
> ...
>
> The vulnerabilities found in the OWASP Juice Shop are categorized into several different classes. Most of them cover different risk or vulnerability types from well-known lists or documents, such as OWASP Top 10 or MITRE's Common Weakness Enumeration. The following table presents a mapping of the Juice Shop's categories to OWASP and CWE (without claiming to be complete).

Category Mappings

Category	OWASP	CWE
Injection	A1:2017	CWE-74
Broken Authentication	A2:2017	CWE-287, CWE-352
Forgotten Content	OTG-CONFIG-004	
Roll your own Security	A10:2017	CWE-326, CWE-601
Sensitive Data Exposure	A3:2017	CWE-200, CWE-327, CWE-328, CWE-548
XML External Entities (XXE)	A4:2017	CWE-611
Improper Input Validation	ASVS V5	CWE-20
Broken Access Control	A5:2017	CWE-22, CWE-285, CWE-639
Security Misconfiguration	A6:2017	CWE-209
Cross Site Scripting (XSS)	A7:2017	CWE-79
Insecure Deserialization	A8:2017	CWE-502
Vulnerable Components	A9:2017	
Security through Obscurity		CWE-656

© license CC BY 3.0. 2026 Henrik Kramselund

Source: Pwning OWASP Juice Shop

Purpose:
Try out some of the described web application flaws in a controlled environment. See how an attacker would be able to gather information and attack through HTTP, browser and proxies.

Suggested method:
Start the web application, start Burp or another proxy - start your browser.

Access the web application through your browser and get a feel for how it works. First step is to register your user, before you can shop.

Don't forget to use web developer tools like the JavaScript console!

Then afterwards find and try to exploit vulnerabilities, using the book from Björn and starting with some easy ones:

Suggested list of starting vulns:

- Admin Section Access the Admin Section
- Error handling Provoke an error
- Forged Feedback Post some feedback in another user's name.
- Access a confidential document
- Forgotten Sales Backup Access a salesman's forgotten backup file.
- Retrieve a list of all user credentials via SQL Injection

Hints:
The complete guide Pwning OWASP Juice Shop written by Björn Kimminich is available as PDF which you can buy, or you can read it online at:
`https://pwning.owasp-juice.shop/companion-guide/latest/`

Solution:
You decide for how long you want to play with JuiceShop.

Do know that some attackers on the internet spend all their time researching, exploiting and abusing web applications.

Discussion:
The vulnerabilities contained in systems like JuiceShop mimic real ones, and do a very good job. You might not think this is possible in real applications, but there is evidence to the contrary.

Using an app like JS instead of real applications with flaws allows you to spend less on installing apps, and more on exploiting.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 9

⚠ Truncate and Encoding Attacks JuiceShop up to 40min

Objective:
Try out some of the problems described in the Juice Shop book using active methods.

Purpose:
The book describes problems with encodings but it can feel a bit fluffy unless you try and see for yourself. We have the JuiceShop which has errors similar to these.

Suggested method:
There is an error in the JuiceShop that can be abused for reading files using encoding `%2500`.

Try to download the file http://localhost:3000/ftp/eastere.gg

It should be possible to even retrieve the content of a file like `C:\Windows\system.ini` or `/etc/passwd` from the server and see if you can read a file. If they exist. This is related to XXE attacks, and seems hard to get working.

Spoiler alert next page!

© license CC BY 3.0. 2026 Henrik Kramselund

Hints:
It's OK to use the solution and work through the example. `http://localhost:3000/ftp/eastere.gg%2500.md`

Also make sure to use `NODE_ENV=unsafe` flag when running docker if you want to try the XXE vuln!

```
export NODE_ENV=unsafe
docker run --rm -p 0.0.0.0:3000:3000 bkimminich/juice-shop
```

Solution:
When you feel you understand the problem of encoding/decoding and sending XML files to an application, reading files, you are done.

Discussion:
Another problem is the filtering done in applications.

In the JuiceShop we can access using URLs like this on the About Us page:
`http://localhost:3000/ftp/legal.md?md_debug=true`

Consider if the URL would match on .md and we were able to send a large filename ending in `loongfilename.md`, but when truncated cut off exactly the `.md` part so we referenced another file.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 10

⚠ JuiceShop Login 15 min

```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}'
AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`,
{ model: models.User, plain: true })
```

Objective:
Try to find the JuiceShop login box implementation, we know it is vulnerable to SQL injection.

Purpose:
Seeing bad code is a design pattern – anti-pattern.

Suggested method:
Find the source code, look for the database lookups.

Hints:
The JuiceShop software is open source, and available at GitHub:
https://github.com/juice-shop/juice-shop

Git clone and searching locally might give the best results.

In this case we can search for `SELECT * FROM`, using a simple tool like grep:

```
user@Projects:juice-shop$ grep -ril SELECT | egrep -v "test|frontend|static"
...
REFERENCES.md
routes/vulnCodeFixes.ts
routes/search.ts
routes/login.ts // oohhhh looks interesting
routes/countryMapping.ts
routes/vulnCodeSnippet.ts
config/oss.yml
config/mozilla.yml
config/default.yml
README.md
```

Solution:
When you have found examples of the database lookups, you are done. See also discussion below though.

Discussion:
Think about how this could be changed. How much would it require to change this into prepared statements? Also, having good source code tools helps a lot! Finding problems, getting an overview of code etc.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 11

⚠ Git hook 30 min

Objective:
Try using a Git hook locally on your workstation, to prevent something.

Purpose:
Running Git with hooks will allow you to perform actions when adding source code. For security we can prevent you from adding something to the source tree which breaks policies we agreed.

First read the documentation:
https://git-scm.com/book/en/v2/Customizing-Git-Git-Hooks

Note: today we only use client side hooks, for better security we should add hooks on the server side.

Using our existing repository:

```
user@Projects:projects$ git clone https://codeberg.org/kramse/kramse-labs.git
Cloning into 'kramse-labs'...
user@Projects:projects$ cd kramse-labs/
user@Projects:kramse-labs$ cd .git/hooks
user@Projects:kramse-labs/.git/hooks$ cat pre-commit.sample
// Look at this file, and activate it using:
user@Projects:kramse-labs/.git/hooks$ cp pre-commit.sample pre-commit
```

Back in the repository try adding a new file with bad name:

```
user@Projects:kramse-labs$
user@Projects:kramse-labs$ touch henrikÆØÅ
user@Projects:kramse-labs$ git add henrikÆØÅ
user@Projects:kramse-labs$ git commit -m "Adding henrikÆØÅ"
Error: Attempt to add a non-ASCII file name.
This can cause problems if you want to work with people on other platforms.
To be portable it is advisable to rename the file.

If you know what you are doing you can disable this check using:
  git config hooks.allow
```

Suggested method:
Run the program Git from your Debian Linux VM

Hints:
Many hooks will depend on the language and what your policy states. Some hooks will be easy to implement, others will require others to agree. Think tabs vs spaces which will forever stay unresolved.

Solution:
When you have tried adding a file with a bad name, and gotten an error, you are done. Feel free to experiment more with the hook.

Discussion:
Many examples can be found on the internet.

Example links:

- https://blogs.vmware.com/opensource/2020/02/03/git-repo-pre-commit-hooks/ which links to the next one
- A framework for managing and maintaining multi-language pre-commit hooks.
  https://pre-commit.com/
- Blog showing how to use this framework for checking a Kubernetes file:
  https://thechief.io/c/hebaeid/how-to-start-left-with-security-using-git-pre-commit-hooks

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 12

⚠ Small programs with data types 15min

Objective:
Try out small programs similar to:

```c
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv)
{       (void) argc;  (void) argv;
                short int i1 = 32767;
                printf("First debug int is %d\n", i1);
                i1++;
                printf("Second debug int is now %d \n", i1);
}
```

```
user@Projects:programs$ gcc -o int1 int1.c && ./int1
First debug int is 32767
Second debug int is now -32768
```

Purpose:
See actual overflows when going above the maximum for the selected types.

Suggested method:
Compile program as is. Run it. See the problem.

Then try changing the int type, try with signed and unsigned. Note differences

Hints:
Use a calculator to find the maximum, like 2^{16}, 2^{32} etc.

Solution:
When you have tried adding one to a value and seeing it going negative, you are done.

Discussion:

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 13

⚠ Pointers and Structure padding 30min

Objective:
Look at some real code from Suricata and Zeek, note how they prevent structure padding.

Purpose:
These applications, usually used for security, dissect raw packets which cannot be trusted.

Suggested method:
Download the source for some software - either of:

- Zeek from `https://zeek.org/get-zeek/`
- Suricata from `https://www.openinfosecfoundation.org/download/`

Unpack using `tar zxf` and use an editor to look up DNS or other packets.

Hints:
DNS is a complex protocol, but looking at the header files should give you an idea. Try going into `src` and doing `less *dns*.h` or use an editor.

Solution:
When you have seen the code for a few `struct` you are done.

Notice structs with `__attribute__((__packed__))`. Note: This ensures that structure fields align on one-byte boundaries - on all architectures.

Maybe also investigate the rest of the file `decode-vxlan.c` if you downloaded Suricata.

Discussion:
The manual for the GNU Compiler Collection (GCC) can be found at:
`https://gcc.gnu.org/onlinedocs/gcc-5.2.0/gcc/Type-Attributes.html`

> packed
> This attribute, attached to struct or union type definition, specifies that each member (other than zero-width bit-fields) of the structure or union is placed to minimize the memory required. When attached to an enum definition, it indicates that the smallest integral type should be used.

Bonus, can we find some structs missing this?

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 14

⚠ Trying PMD static analysis 30 min

Objective:
Try the program PMD locally on your workstation

Purpose:
Running PMD will allow you to use static analysis for code.

Suggested method:
Run the program from your Debian Linux VM, this tool is free and easy to get running. It uses Java, so if you like run it on your Windows or Mac instead.

Follow instructions from the Getting Started
`https://pmd.github.io/latest/pmd_userdocs_installation.html`

The download is from latest, so check the releases page: https://github.com/pmd/pmd/releases

```
$ sudo apt install openjdk-17-jre
$ cd $HOME
$ wget https://github.com/pmd/pmd/releases/download/pmd_releases/6.49.0/pmd-bin-6.49.0.zip
$ unzip pmd-bin-6.49.0.zip
$ alias pmd="$HOME/pmd-bin-6.49.0/bin/run.sh pmd"
$
```

Note: this only creates the alias `pmd` for this session. To make this more permanent, you could add this to a profile like `.bashrc`

Next get some source code and run PMD:

```
$ git clone --branch rel/2.17.2 https://gitbox.apache.org/repos/asf/logging-log4j2.git
... downloads the source code for log4j
$ pmd -d logging-log4j2 -R rulesets/java/quickstart.xml -f text
```

Hints:
PMD uses Java, so there should be a JDK/JRE on the system, I install the one from OpenJDK above.

You may need to adjust the version numbers for the JDK/JRE, PMD and select another version of log4j to download.

Solution:
When you have gotten a run of PMD going, you are done.

Discussion:
Doing the above probably outputs more than 4500 lines from the PMD program!

How would you proceed?

Some are tedious but easy to fix, like Unused import – importing some library which is not really used. Things like empty method, empty catch block etc. may be source missing.

First time use of a new tool will probably find a LOT.

If you are using Maven you could also use their reporting
https://maven.apache.org/plugins/maven-pmd-plugin/project-reports.html

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 15

⚠ RBAC Access permissions on GitHub 15 min

Objective:
See the actual permissions in your own organisation – a real life example of role based access control (RBAC).

Purpose:
GitHub is a very popular code sharing site, and whoever has write or admin access to a repository can change the code, the workflows and often the deployment. Most organisations have more people with high permissions than they think.

Suggested method:
Open your organisation on GitHub, or a repository you work on, and look at the people, teams and settings. If you do not have access to an organisation, use the documentation and a repository of your own.

Questions to answer:

- Who are organisation owners? Owners have admin access to every repository in the organisation
- How many people or teams have Write, Maintain or Admin on the repository?
- Are there outside collaborators – do they still need access?
- Is the default branch protected – can anyone push directly to it?
- Are there deploy keys – who has the private key?

Hints:
GitHub repository roles, from least to most access: Read, Triage, Write, Maintain, Admin. The documentation has a table of what each role can do, including who can create and change GitHub Actions secrets and workflows:
https://docs.github.com/articles/repository-permission-levels-for-an-organization

Note: someone holding the private key of a deploy key can keep reading from or writing to the repository, even after being removed from the organisation.

Solution:
When you have looked at the real permissions for at least one repository and compared them with what you expected, you are done.

Discussion:
Discuss with your neighbour: would you have expected this? Who reviews these permissions, and how often?

The internet is decentralized, but recent years see more centralization – GitHub, DNS with Google DNS, Cloudflare. What are some problems in this?

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 16

⚠ Research CSRF protection in Django and OWASP 15 min

Objective:
See how a mature framework implements Cross-Site Request Forgery (CSRF) protection, and compare with the patterns recommended by OWASP.

Purpose:
Creating anti-CSRF tokens is best left to existing middleware. The OWASP CSRF Prevention Cheat Sheet starts by recommending: check if your framework has built-in CSRF protection, and use it.

Suggested method:
Read the Django description of its CSRF protection:
https://docs.djangoproject.com/en/stable/ref/csrf/

Then read the overview section of the OWASP CSRF Prevention Cheat Sheet:
`https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html`

The cheat sheet describes these patterns, among others:

- Synchronizer token pattern – for stateful software, the token is stored in the session
- Double submit cookie – for stateless software
- Fetch Metadata headers – for software targeting modern browsers
- SameSite cookies and checking the Origin header – as defense in depth

Hints:
Django has a LOT of security features built-in and is a good example, even if you do not use Python. Look at how the protection is enabled, and how a developer can turn it off for a single view.

Then check the framework you use yourself: Spring, ASP.NET Core, Express, Angular, ...

Solution:
When you can say which OWASP pattern Django uses, and whether your own framework has CSRF protection enabled by default, you are done.

Discussion:
Where in your own code base would something like `@csrf_exempt` hide? Could you search for it in a Git hook or in CI?

Note: an XSS vulnerability can be used to defeat token based CSRF defenses – so XSS prevention is also CSRF prevention.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 17

⚠ Kyverno Playground 15 min

Objective:
Read and try Kubernetes policies written for the Kyverno policy engine – without installing anything.

Purpose:
Policy engines like Kyverno run as admission controllers in Kubernetes and can validate, mutate and generate resources and verify image signatures. They allow you to enforce rules like "containers must not run as root" before anything runs in the cluster. The Kyverno Playground simulates Kyverno in the browser.

Suggested method:
Open the Kyverno Playground:
https://playground.kyverno.io/

Follow the built-in tutorial. Then load a policy from the catalog, for example from the Pod Security Standards category, and test it against a pod.

Hints:
Use this pod, similar to the Juice Shop example from day 0, as resource:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: juice-shop
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: juice-shop
    image: bkimminich/juice-shop
    securityContext:
      allowPrivilegeEscalation: false
      capabilities:
        drop: ["ALL"]
```

Does it pass? Now remove `runAsNonRoot`, or add `privileged: true` – what happens?

The policy library has many ready-made policies, with categories and descriptions:
https://kyverno.io/policies/

Solution:
When you have evaluated at least one policy against a resource that passes and one that fails, you are done.

Discussion:
Policies can run in audit mode, reporting violations, or enforce mode, rejecting resources. Would you enforce from day one, or start with audit?

Kubernetes has a built-in Pod Security Admission controller for the Pod Security Standards. When do you need a policy engine in addition? Think about rules like "only images from our own registry" or

“images must be signed”.

The Kyverno CLI can apply the same policies in CI – before the resource ever reaches the cluster.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 18

⚠ Threat-model your pipeline 15 min

Objective:
Think about how to secure the build and deployment pipeline itself – not just which tools to add to it.

Purpose:
A CI/CD pipeline can read the source code, holds secrets and can deploy to production. It is a production system and an attractive target. The tj-actions/changed-files compromise in March 2025 (CVE-2025-30066) showed how a third-party GitHub Action with moved version tags could leak secrets from more than 23,000 repositories.

Suggested method:
Take one of your own pipelines, alone or with your neighbour, and answer:

- Who can change the pipeline/workflow files – is that reviewed like other code?
- Which secrets can a job reach – and can a pull request from a fork reach them?
- Are third-party actions, plugins and images pinned to a commit SHA or digest, or to a tag that can be moved?
- Where do the jobs run – shared runners, self-hosted runners reused between jobs?
- If you added a scanner aggregator like MegaLinter tomorrow: how many tools would you implicitly trust, who updates them, and where do the results go?
- What would happen if one of these tools was compromised, like tj-actions?

Hints:
MegaLinter runs many linters and security scanners in one container, see the security flavor: https://megalinter.io/

CISA alert about the tj-actions compromise:
https://www.cisa.gov/news-events/alerts/2025/03/18/supply-chain-compromise-third-party-tj-actionschanged-files-cve-2025-30066-and-reviewdogaction

Solution:
When you have answered the questions for one pipeline and found at least one thing you would change, you are done.

Discussion:
The answers are mostly about permissions, pinning and isolation – not about which scanner to add. Least privilege, separation and logging apply to the pipeline just like everywhere else.

What is the trade-off with pinning? Pinned dependencies must be updated deliberately – tools like Dependabot and Renovate can update pins for you.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 19

⚠ npm audit and osv-scanner on Juice Shop 30 min

Objective:
Check the dependencies of a real application for known vulnerabilities and known malicious packages, using `npm audit` and `osv-scanner`.

Purpose:
Most of the code in a modern application is dependencies. This exercise shows how many there are, how many have known vulnerabilities, and how the way you install them matters for supply chain attacks like the npm compromises in September 2025.

Suggested method:
You need Git and Node.js with npm. If you do not have Node.js installed, run it in a container – the official `node` images include Git:

```
docker run --rm -it node:22 bash
```

Part 1:

```
git clone --depth 1 https://github.com/juice-shop/juice-shop.git
cd juice-shop
cat .npmrc
npm install --package-lock-only --package-lock=true --ignore-scripts
npm audit
```

Then install osv-scanner, a single binary from the GitHub releases page – pick the file for your platform: https://github.com/google/osv-scanner/releases

```
./osv-scanner scan source -L package-lock.json
```

Part 2, after the break:

- Compare your `npm audit` result with your neighbour – is it the same number?
- Find `debug` and `chalk` in `package-lock.json` – which versions?
- What would `npm install` have given you on 8 September 2025?
- What protects you: a committed lockfile, `npm ci`, `--ignore-scripts`, waiting a few days before taking new versions?

Hints:
Juice Shop does not commit a lockfile – its `.npmrc` contains `package-lock=false`. The command above creates one without installing anything into `node_modules`.

`--ignore-scripts` is important: a normal `npm install` runs the lifecycle scripts of the packages, like `postinstall`. This is how the Shai-Hulud worm ran on developer machines and CI runners in September 2025.

When tested in October 2026 the lockfile had around 1,350 packages, and `npm audit` reported 46 vulnerabilities (2 low, 17 moderate, 18 high, 9 critical). Your numbers will differ.

© license CC BY 3.0. 2026 Henrik Kramselund

The `frontend/` directory has its own `package.json` – you can do the same there.

Solution:
When you have run both `npm audit` and `osv-scanner` and can explain one finding, you are done.

Discussion:
Without a committed lockfile, the version ranges in `package.json` resolve to the newest matching versions at the time of install. Two people running the same command on different days get different dependency trees. This is why the result changes from day to day – and why anyone running `npm install` on 8 September 2025 could get the malicious `debug` or `chalk`.

Juice Shop is deliberately vulnerable, so many findings are expected.

`npm audit` uses the GitHub Advisory Database. `osv-scanner` uses OSV, which also contains known malicious packages from the OpenSSF Malicious Packages repository, with IDs starting with `MAL-`. Note that both only report a malicious package if your lockfile contains the exact malicious version.

Bonus question: how would you find out whether any of the packages in your tree was compromised at some point in time, even if your version is clean?

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 20

❶ Identify Session Tokens 30 min

```
Cookie: login=CustomerId=900180&LanguageId=9; OldBrowser=%220%22; Pool=rosalina;
ShowNativeLogin=true; DebugCustomerId=900180; DebugPersonId=400954;
OnboardingCompletedFeatures=h=1c744782963a2478d5db92a9981d401a&ofc_47=True&ofd_49=True;
ASP.NET_SessionId=u0245fara4x3qmkli5refddg;...
```

Objective:
Look at a real application and identify session tokens.

Verify session settings, like Anti-XSS and Anti-CSRF tokens, if present.

Purpose:
Web applications are faking sessions, each request are independent by design of the protocol. This is done using session cookies and similar methods.

Running Burp while performing a login will allow you to look into session identifiers.

Running a test using Mozilla Observatory first will allow you to analyse the settings for the web site.

Some things to look for

- Cookie settings, Secure Flag and http-only, SameSite
- HSTS header `https://en.wikipedia.org/wiki/HTTP_Strict_Transport_Security`
- Random – how to check that?

Suggested method:
Use the Juice Shop with Burp from the day 0 exercises, or the developer tools in your browser, and log in to an application you use – preferably one of your own in a test environment.

Hints:
Look in the header section, and look for `Cookie:`. Common ones are `ASP.NET_SessionId`, `JSESSIONID` or `PHPSESSID`. Juice Shop uses a JWT – decode it, what is in the payload?

Solution:
When you have identified session cookies or tokens and checked the settings using a scanner, test web site or similar you are done. It is recommended though to dive a bit into the application and how these are used.

Discussion:
Most sites today have switched to using HTTPS, but some are not according to best practice. To prevent the use of credentials or cookies over insecure connections, we should not allow calls to happen over HTTP.

Various tools like OWASP Zap and Burp suite can also be used for analyzing the session cookies, for randomness/entropy:
https://portswigger.net/burp/documentation/desktop/tools/sequencer/getting-started

Checkout the story of the old extension Firesheep on wikipedia, what could happen when session cookies were not protected:
https://en.wikipedia.org/wiki/Firesheep

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 21

ℹ Postman API Client 20 min

Objective:
Get a program capable of sending REST HTTP calls installed.

Purpose:
Debugging REST APIs is often needed, including testing authentication with tokens.

Suggested method:
Download the app from https://www.postman.com/downloads/

Available for Windows, Mac and Linux.

Hints:
Do not run Postman as root. A free account is optional, you can continue without one for basic features.

Command line alternatives are `curl` and `jq`, which work everywhere.

Solution:
When you have performed a REST call from within this tool, you are done.

Example: use the fake site https://jsonplaceholder.typicode.com/todos/1 and other similar methods from the same (fake) REST API

If you have Juice Shop running, try its REST API, for example the product search at `/rest/products/search?q=`

Discussion:
Multiple applications and plugins can perform similar functions. This is a standalone app.

Where do tools like this store your tokens and API keys? Are they synchronised to a cloud account?

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 22

❶ PHP Passwords 30 min

```php
<?php
/**
 * We just want to hash our password using the current DEFAULT algorithm.
 * This is presently BCRYPT, and will produce a 60 character result.
 *
 * Beware that DEFAULT may change over time, so you would want to prepare
 * By allowing your storage to expand past 60 characters (255 would be good)
 */
echo "Password: henrik42 hash: ";
echo password_hash("henrik42", PASSWORD_DEFAULT);
?>
```

Objective:
Try the PHP function `password_hash` on the same password with different algorithms and compare hashes

Purpose:
Running the PHP `password_hash` function will enable you to use a quality implementation of hashing algorithms.

Suggested method:
Run the program using PHP on the command line, using the documentation https://www.php.net/manual/en/function.password-hash.php

If you do not have PHP installed, a container works: `docker run --rm -it php:cli bash`

Hints:
Create a program that hash a password using multiple algorithms.

You can ask the user for input, or hard-code in the program source.

The output is expected similar to:

```
$ php my_passwordprogram.php
The password is: henrik42
The password hash using default is: $2y$10$.fwefw...2kljhfwe
... print all of the hashes possible
```

Solution:
When you have a PHP program that can hash a password you are done.

Also, explain why above example start with `$2y$10$`

Discussion:
There are multiple tunable parameters for some algorithms. Feel free to experiment with them and compare output.

Note the length of output from the hashing algoritms do NOT say how strong they are. An algorithm which output less can be more secure than one with longer output.

© license CC BY 3.0. 2026 Henrik Kramselund

Password hashing is not easy, and consider an environment with 100s of applications!

Which function does your own framework use for password hashing?

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 23

❶ Django String Handling 20min

Recommendations for handling strings, how does Python help, how does Django handle strings, and input validation

Objective:
Look into string handling in Django framework

Purpose:
See that Python 3 and Django includes functions for conversion, so you dont need to write these yourself.

Suggested method:
First look into Python3 string handling, for example by looking at
`https://docs.python.org/3/library/text.html`

Then look at Django string and unicode handling:

- Look for string, url, encode, decode in
  `https://docs.djangoproject.com/en/stable/ref/utils/`
- `https://docs.djangoproject.com/en/stable/ref/unicode/`

Hints:
Follow the URLs above.

Solution:
When you have looked up and seen the names of a few relevant functions like these below, you are done:

```
django.utils.html escape(text)
django.utils.safestring
django.utils.dateparse
```

Note the links where you can see the source implementation of the functions.

Discussion:
Are strings easy to work with?

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 24

ℹ Django ORM 20 min

```python
from django.db import models

class Person(models.Model):
    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)
```

```sql
CREATE TABLE myapp_person (
    "id" serial NOT NULL PRIMARY KEY,
    "first_name" varchar(30) NOT NULL,
    "last_name" varchar(30) NOT NULL
);
```

Objective:
See how a mapping model, Objectrelational mapping (ORM) can help reduce complexity for you.

Purpose:
Using an ORM frees you from a lot of low level detail, and keeps your application more portable between database systems. It also uses parameterized queries, preventing most SQL injection.

Suggested method:
Read about the Django Objectrelational mapping (ORM) at:

- https://docs.djangoproject.com/en/stable/topics/db/models/
- https://docs.djangoproject.com/en/stable/topics/db/queries/

Hints:
Many programming languages use ORM and have similar functions. See Wikipedia for a list:
`https://en.wikipedia.org/wiki/List_of_object-relational_mapping_software`

Solution:
When you have read about either the Django model, or a similar method in another framework or programming language you are done.

Discussion:
Look for the escape hatches: how can a developer write raw SQL in Django, and how do you make that safe?

I have myself used Grails, A powerful Groovy-based web application framework for the JVM built on top of Spring Boot

https://grails.org/

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 25

❶ Django email validation 30 min

Objective:
Find a mature implementation for validating email, a common requirement in modern applications.

We will use Django as an example.

Purpose:
See if we can find an implementation that will suit our own projects, even if not using Django.

Suggested method:
Find the Django email validation – how is the validation done, can it be copied and reused somewhere else.

Hints:
The Django software is open source, and available at:
https://github.com/django/django

Git clone and searching locally might give the best results.

```
$ git clone --depth 1 https://github.com/django/django.git
$ cd django
$ grep -ril email | less
```

One file includes `class EmailValidator:` which sounds promising.

Solution:
When you have found the files implementing the actual email validation, not all related files, only the one doing the validation – you are done.

Discussion:
Email addresses are notoriously hard to validate, since the standard is very complex. Often we can do with less, say if we want to use it as a user-id. Then we might decide NOT to support comments and things like `hlk+test@kramse.org`

Which other validators would be nice to have, in your own library?

Via Angular: `https://v17.angular.io/api/forms/Validators#email`
`https://html.spec.whatwg.org/multipage/input.html#valid-e-mail-address`

Exercise 26

❶ Try American fuzzy lop up to 60min

Objective:
Try a fuzzer. We will use the popular american fuzzy lop named after a breed of rabits, today maintained as the fork AFL++.

Purpose:

> American fuzzy lop is a security-oriented fuzzer that employs a novel type of compile-time instrumentation and genetic algorithms to automatically discover clean, interesting test cases that trigger new internal states in the targeted binary. This substantially improves the functional coverage for the fuzzed code. The compact synthesized corpora produced by the tool are also useful for seeding other, more labor- or resource-intensive testing regimes down the road.

Source: http://lcamtuf.coredump.cx/afl/

Suggested method:
The original AFL is no longer developed. Use AFL++ which is packaged in many Linux distributions, e.g. `apt install afl++` on Debian, and follow the tutorials:
https://github.com/AFLplusplus/AFLplusplus

Later if you like, fuzz the small C programs from the day 0 exercises.

Hints:
Look at the many projects which have been tested by AFL, the bug-o-rama trophy case on the original web page.

Solution:
When AFL++ is installed and has run a fuzzing session against a program - no matter if it found anything.

Discussion:
For how long is it reasonable to fuzz a program? A few days - sure. Maybe run multiple sessions in parallel!

Could fuzzing be part of your CI pipeline? Some projects run continuous fuzzing, like Google OSS-Fuzz.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 27

❶ Real Vulnerabilities up to 30min

Objective:
Look at real vulnerabilities. Choose a few real vulnerabilities, prioritize them.

Purpose:
See that the error types described in the books - are still causing problems.

Suggested method:
We will use the 2019 Exim errors as examples. Download the descriptions from:

- Exim RCE CVE-2019-10149 June
  `https://www.qualys.com/2019/06/05/cve-2019-10149/return-wizard-rce-exim.txt`
- Exim RCE CVE-2019-15846 September
  `https://exim.org/static/doc/security/CVE-2019-15846.txt`

When done with these think about your own dependencies. What software do you depend on? How many vulnerabilities and CVEs are for that?

I depend on the OpenBSD operating system, and it has flaws too:
`https://www.openbsd.org/errata.html`

You may depend on OpenSSH from the OpenBSD project, which has had a few problems too:
`https://www.openssh.com/security.html`

Hints:
Remote Code Execution can be caused by various things, but most often some kind of input validation failure.

Solution:
When you have identified the specific error type, is it buffer overflows? Then you are done.

Discussion:
How do you feel about running internet services. Lets discuss how we can handle running insecure code.

What other methods can we use to restrict problems caused by similar vulnerabilities.

A new product will often use a generic small computer and framework with security problems.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 28

ℹ Security.txt 15 min

Objective:
Learn how to contact sites with security problems, through `security.txt` files.

Purpose:
Contacting a site which has a security problem can be a difficult task. Sending information through the usual support often requires a contract, and you are helping the company?!

Also, how do you make sure people can contact YOUR organisation with security issues in your sites. Receiving vulnerability reports is the incoming side of vulnerability management.

> Summary
> When security risks in web services are discovered by independent security researchers who understand the severity of the risk, they often lack the channels to disclose them properly. As a result, security issues may be left unreported. security.txt defines a standard to help organizations define the process for security researchers to disclose security vulnerabilities securely.
>
> security.txt files have been implemented by Google, Facebook, GitHub, the UK government, and many other organisations. In addition, the UKs Ministry of Justice, the Cybersecurity and Infrastructure Security Agency (US), the French government, the Italian government, and the Australian Cyber Security Centre endorse the use of security.txt files.

Source: https://securitytxt.org/

Suggested method:
Run the wizard on the site to create a template file for your own purposes.

Hints:
Looking at other sites can help, since they have run these for some time. So find a few examples, the web site mentions a few. The file is found at `/.well-known/security.txt`

Solution:
When you have seen a few security.txt files, and possibly created one yourself, you are done.

Discussion:
Does your organisation have one? Who reads the mailbox it points to?

Before this initiative there was a common set of email addresses, used for specific purposes. I still recommend them to customers:

```
MAILBOX       AREA                USAGE
-----------   -----------------   --------------------------
ABUSE         Customer Relations  Inappropriate public behaviour
NOC           Network Operations  Network infrastructure
SECURITY      Network Security    Security bulletins or queries
...
MAILBOX       SERVICE             SPECIFICATIONS
-----------   -----------------   --------------------------
POSTMASTER    SMTP                [RFC821], [RFC822]
HOSTMASTER    DNS                 [RFC1033-RFC1035]
WEBMASTER     HTTP                [RFC 2068]
WWW           HTTP                Synonym for WEBMASTER
```

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 29

ℹ Securing the JuiceShop

Objective:
Layout a plan for securing the Juice Shop

Purpose:
Lets discuss how we can proceed if JuiceShop was a real shop in our organisation

Suggested method:
Break down the immediate steps for securing this shop.

Should we go and buy a security product for filtering requests?

Should we start logging all requests and analyzing them?
To see when we are attacked

Use the final recommendations from day 1 as a checklist: dependencies, authentication, container hardening, policies, pipeline, separation of environments, logging and monitoring.

Hints:
There are some gaping holes that can be removed, files that could be downloaded.

Some functions are old and can be removed or turned off.

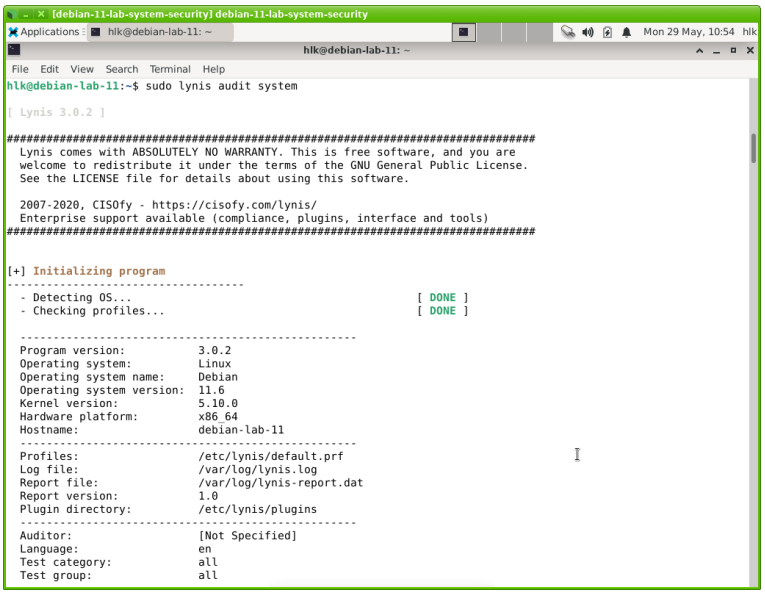
Solution:
There is no solution, the discussion is the solution.

Discussion:
There are some things that can be fixed in production, and some can't easily be redone without major interruption

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 30

❶ Lynis Auditing, System hardening, and Compliance testing 20min



Objective:
Try out the Lynis tool for scanning your local Unix server.

Purpose:
Lynis is a quick tool to run, and gives concrete advise for things to change. It is the host level equivalent of kube-bench from day 0.

Suggested method:
Install using APT on a Debian or Ubuntu system, a virtual machine or a test server, and run the tool

```
sudo apt install lynis
sudo lynis audit system
```

Hints:
Use the web site for the tool Lynis `https://cisofy.com/lynis/` and audit your system

Solution:
When you have run the tool you are done. Better if you have browsed some of the output

Discussion:
Which suggestions would you apply to a production server – and which would you put in your base image or configuration management, so every new server gets them?

© license CC BY 3.0. 2026 Henrik Kramselund